1 — What You Built Today

You wrote **emitter.py** — the third and most important stage of the RTL toolchain. It takes a parsed **ISASpec** object and automatically generates a complete synthesizable Verilog file — **decoder.v** — without any manual RTL authoring. This is the core claim of your entire project.

2 — The Key Insight

Generating Verilog is just Python string manipulation. You build a list of strings line by line, then join them with newlines and write to a file. The Verilog structure never changes — only the values inside it change based on your ISA spec.

```
lines = []
lines.append('module decoder(')
lines.append(' input [3:0] opcode,')
# ... more lines ...
output = '\n'.join(lines) # join into one string
with open('decoder.v', 'w') as f:
f.write(output) # write to file
```

3 — VerilogEmitter Class Structure

```
class VerilogEmitter:
def __init__(self, spec, output_dir):
self.spec = spec
self.output_dir = output_dir

def emit_decoder(self):
lines = []
# 1. Header comments
# 2. Module declaration with ports
# 3. always @(*) begin
# 4. case (opcode) — one block per instruction
# 5. default case
# 6. endcase, end, endmodule
# 7. Write to file
```

4 — Generating the Module Declaration

The module ports are generated dynamically from the spec — no hardcoded widths or signal names:

```
lines.append('module decoder(')
lines.append(' input [' + str(self.spec.opcode_bits-1) + ':0] opcode,')

for i, sig in enumerate(self.spec.ControlSignals):
is_last = (i == len(self.spec.ControlSignals) - 1)
comma = '' if is_last else ','
if sig.width > 1:
line = ' output reg [' + str(sig.width-1) + ':0] ' + sig.name + comma
else:
line = ' output reg ' + sig.name + comma
lines.append(line)

lines.append(');')
```

5 — Generating the Case Statement

The nested loop is the heart of the emitter — outer loop over instructions, inner loop over control signals:

```
    for n in self.spec.Instructions:
    lines.append('  4\'b' + n.opcode + ': begin // ' + n.mnemonic)
    for i, m in enumerate(self.spec.ControlSignals):
    lines.append('    ' + m.name + ' = '
    + str(m.width) + '\'b' + n.control[m.name] + ';')
    lines.append('  end')

    # default case
    lines.append('  default: begin')
    for sig in self.spec.ControlSignals:
    lines.append('    ' + sig.name + ' = '
    + str(sig.width) + '\'b' + '0' * sig.width + ';')
    lines.append('  end')
```

6 — Key Python Concepts Used

Concept	How it was used
`list + join`	Build Verilog line by line, join with \n at the end
`enumerate`	Get both index and item when looping: for i, sig in enumerate(...)
`is_last check`	i == len(list)-1 detects the last item for comma logic
`'0' * width`	Repeats the string '0' width times — e.g. '0'*3 = '000'
`nested loops`	Outer loop over instructions, inner loop over signals
`with open(...,'w')`	Opens file for writing, auto-closes when done
`\n.join(lines)`	Joins list of strings into one string with newlines

7 — What decoder.v Contains

✓ Module declaration with 1 input (opcode) and 7 outputs (control signals)

✓ always @(*) block — re-evaluates whenever opcode changes

✓ case statement with 14 instruction blocks — one per opcode

✓ Each block assigns all 7 control signals from the ISA spec

✓ default case — all signals zero for undefined opcodes

✓ Compiles clean with iverilog — ready for simulation tomorrow

■ Homework — Before Day 7

→ *Tomorrow is Day 7 — simulation*

→ *You will compile decoder.v with iverilog and run it*

→ *Think about this: how do you test a decoder? What inputs do you feed it?*

→ *Answer: a testbench — a Verilog file that applies each opcode and checks the outputs*